Name: Abasiamek Isaac Amos
EMPLID: 24229842

1) Build instructions
    a) As recommended, I have written the implementation in C++. to compile the program run the following in the project directory where the Makefile is located:
        i) (type) -> make
            (1) This compiles HW0_Isaac_Amos.cpp and crypto.h using g++ with -02 optimization and outputs an executable named hw0.
2) Manual execution instructions
    a) To run the binary directly from the terminal:
        (type) -> ./hw0
            (1) Once running, the program awaits commands from standard input until the end-of-file (press ctrl+d on macOS/Linux to signal EOF).
    b) You can also commands directly into the prompt:

```
setup_params 23 5 11
rsa_encrypt 3233 17 65
rsa_decrypt 3233 2753 2790
dh_key_exchange 8
elgamal_encrypt 8 15
elgamal_decrypt 6 12 18
```

3) Implementation overview
    a) Primitives (crypto.h):
        i) mod_exp: Implemented using repeated squaring (O(log*e*) time complexity) with intermediate __int128 casting to prevent 64-bit overflow during modular multiplication.
        ii) mod_inverse: Implemented using the iterative extended euclidean algorithm to solve (a * x) + (m * y) = 1 and normalized to produce a positive remainder modulo *m*.
    b) Core schemes (crypto.h):
        i) rsa_encrypt / rsa_decrypt: Directly computes modular exponentiations modulo *N*.
        ii) dh_key_exchange: Samples secret key using std::mt19937_64 and computes the shared secret A^kb (mod *p)*.
        iii) elgamal_encrypt / elgamal_decrypt: handles ephemral exponent generation, masking factor computation, and modular inversion unmasking.
    c) Command line interface parser (HW0_Isaac_Amos.cpp): processes newline-delimited command strings sequentially from std::cin until end-of-file.